



Faculty of Engineering & Technology
Electrical & Computer Engineering Department
ENCS 3340, Artificial Intelligent (AI)
Second Semester (2024/2025)

Project 2

Mohammed Shamasneh	Nom. : 1220092	Section: 1
Sari Murad Abdalghani	Nom. :1220982	Section: 4

Instructors: Dr. Yazan Abu Farha

Submission Date: June, 26, 2025

Abstract

In this project, we present a comparative study of three machine learning models: Naive Bayes, Decision Tree, and Feedforward Neural Network (MLP Classifier) for image classification within the same category. A dataset of 651 labeled images, resized to a fixed dimension of 64×64 pixels and flattened into 1D feature vectors, is used for training and testing.

The Naive Bayes classifier, known for its simplicity and efficiency with high-dimensional data, is evaluated using statistical features (mean and standard deviation) extracted from the images. The Decision Tree model leverages pixel intensity values to construct interpretable rule-based classifiers and is visualized through a simplified tree structure. The Feedforward Neural Network (MLP Classifier), used optionally, explores deeper representations using two hidden layers.

All models are evaluated using standard performance metrics: accuracy, precision, recall, F1-score, and confusion matrix. The results are analyzed to highlight the strengths and limitations of each model in terms of classification performance, interpretability, and generalization to unseen data. This study provides valuable insights into model selection for practical image classification tasks.

Dataset and train Description

In our project, we selected the category of **animals** with three classes: **dogs**, **cats**, and **horses**. The dataset consists of a total of **651 images**, distributed as follows:

- Dogs: 241 images
- Cats: 193 images
- Horses: 217 images

Each image was resized to a fixed dimension of 64×64 pixels and converted to RGB format. To prepare the data for machine learning models, each image was then flattened into a 1D feature vector, allowing it to be used as input for the classifiers.



Figure 1: Horses in data set



Figure 2: Cats in data set



Figure 3: Dogs in data set

During training, the dataset was split into 75% for training and 25% for testing (prediction). We used a fixed random state to ensure reproducibility and obtain consistent results across multiple runs.

```
33 #-----| Load and process the images |-----
34
35 def load_images(data_path, img_size=(64,64)): # fixed dimension = 64x64
36     X, y = [], []
37     total = 0 # get total images
38     for label in os.listdir(data_path):
39         label_path = os.path.join(data_path, label)
40
41         for fname in os.listdir(label_path):
42             fpath = os.path.join(label_path, fname)
43             total += 1
44             img = Image.open(fpath).convert('RGB') # convert to RGB format
45             img = img.resize(img_size) #convert to fixed dimension = 64x64
46             X.append(np.array(img).flatten()) # Flatten the image into a 1D array then append to list
47             y.append(label) # name of classification from the name of folder
48
49     print(f"Loaded {total} images from dataset.")
50     return np.array(X), np.array(y)
51
52 X, y = load_images("dataset/", (64,64)) #calls the load_images function
53
54 le = LabelEncoder() # to convert the name of classification to numerical value (encoding)
55 y_encoded = le.fit_transform(y)
56
57 #-----
58
59
60
61 #-----| to train, 75% train and 25% test, the random state to take the same photo at every run |-----
62 X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.25, random_state=42, stratify=y_encoded)
63
```

Figure 4: load images and train code

Model Descriptions

Naïve Bayes

We used simple statistical features (mean and standard deviation) of pixel values from the images. This model assumes feature independence and is known for its simplicity and speed.

```
65 #-----| calculate the mean and standard deviation for Naive Bayes |-----
66
67 def stats_features(X):
68     means = X.mean(axis=1)
69     stds = X.std(axis=1)
70     return np.stack([means, stds], axis=1)
71
72 X_train_stats = stats_features(X_train)
73 X_test_stats = stats_features(X_test)
74
75
76 #-----| Train & Evaluate Models |-----
77
78 metrics_results = {} # dictionary to store performance metrics for each model
79 conf_matrices = {} # dictionary to store confusion matrices for each model
80
81
82 #-----| Naive Bayes (using only mean and standard deviation) |-----
83 nb_model = GaussianNB()
84 nb_model.fit(X_train_stats, y_train) # to train the model
85 y_pred_nb = nb_model.predict(X_test_stats) # predict on the test set
86
87 # Calculate evaluation metrics
88 acc = accuracy_score(y_test, y_pred_nb) # find accuracy
89 prec = precision_score(y_test, y_pred_nb, average='macro') # find precision
90 rec = recall_score(y_test, y_pred_nb, average='macro') # find recall
91 f1 = f1_score(y_test, y_pred_nb, average='macro') # find F1
92 cm = confusion_matrix(y_test, y_pred_nb) # find confusion
93 metrics_results["Naive Bayes"] = [acc, prec, rec, f1] # store the value
94 conf_matrices["Naive Bayes"] = cm # store the confusion
```

Figure 5: Naïve Bayes

Decision Tree

We used all pixel intensity values as features, and train and test of full depth, allowing it to capture detailed patterns in the data, and we visualize a simplified version of the tree with a max depth of 3, which helps in understanding the decision-making process and how the model distinguishes between classes based on pixel values.

```
99 #-----| Decision Tree (using all pixel values) |-----
100
101 dt_model = DecisionTreeClassifier(random_state=42)
102 dt_model.fit(X_train, y_train) # Train the full-depth tree
103
104 # Create and plot a simplified tree (max depth = 3) for visualization
105 dt_model_simple = DecisionTreeClassifier(random_state=42, max_depth=3)
106 dt_model_simple.fit(X_train, y_train)
107 plt.figure(figsize=(18,8))
108 tree.plot_tree(dt_model_simple, filled=True, feature_names=None, class_names=le.classes_)
109 plt.title("Decision Tree (max_depth=3)")
110 plt.show()
111
112
113 # Predict and evaluate the full-depth tree
114 y_pred_dt = dt_model.predict(X_test) # Predict on the test set
115 acc = accuracy_score(y_test, y_pred_dt) # find accuracy
116 prec = precision_score(y_test, y_pred_dt, average='macro') # find precision
117 rec = recall_score(y_test, y_pred_dt, average='macro') # find recall
118 f1 = f1_score(y_test, y_pred_dt, average='macro') # find F1
119 cm = confusion_matrix(y_test, y_pred_dt) # find confusion
120 metrics_results["Decision Tree"] = [acc, prec, rec, f1] # store the value
121 conf_matrices["Decision Tree"] = cm # store the confusion
122
```

Figure 6: Decision Tree

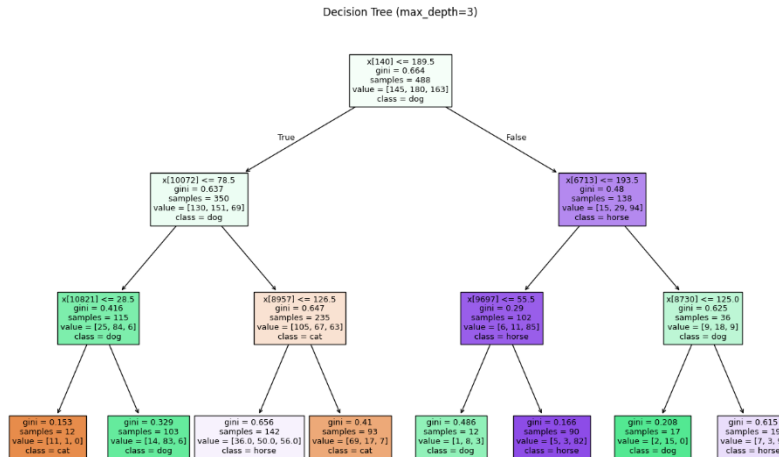


Figure 7: Decision Tree with max depth of 3

Feedforward Neural Network (MLP Classifier)

We implemented a two-layer Feedforward Neural Network, with hidden layer sizes of (32, 128). The network was trained on the full pixel features to capture detailed spatial patterns within the images.

This model uses non-linear activation functions to learn complex relationships between the input features (pixel intensity values) and the output labels (classes). By leveraging multiple hidden layers, the MLP is capable of extracting deeper feature representations compared to simpler models like Naive Bayes and Decision Tree.

```

126 #-----| MLP Classifier (all pixels) |-----
127
128 mlp_model = MLPClassifier(hidden_layer_sizes=(32, 128), max_iter=200, random_state=42)
129 mlp_model.fit(X_train, y_train) # Train the MLP model
130 y_pred_mlp = mlp_model.predict(X_test) # Predict on the test set
131
132 acc = accuracy_score(y_test, y_pred_mlp) # find accuracy
133 prec = precision_score(y_test, y_pred_mlp, average='macro') # find precision
134 rec = recall_score(y_test, y_pred_mlp, average='macro') # find recall
135 f1 = f1_score(y_test, y_pred_mlp, average='macro') # find F1
136 cm = confusion_matrix(y_test, y_pred_mlp) # find confusion
137 metrics_results["MLP Classifier"] = [acc, prec, rec, f1] # store the value
138 conf_matrices["MLP Classifier"] = cm # store the confusion
139

```

Figure 8: Feedforward Neural Network (MLP Classifier)

Results

The Naive Bayes model shows significant confusion between the three classes. Many cat images were predicted as dogs or horses, and similar misclassifications occurred with horse images. This poor performance is expected, as the model uses only simple statistical features and assumes independence between pixels, which limits its ability to capture image complexity.

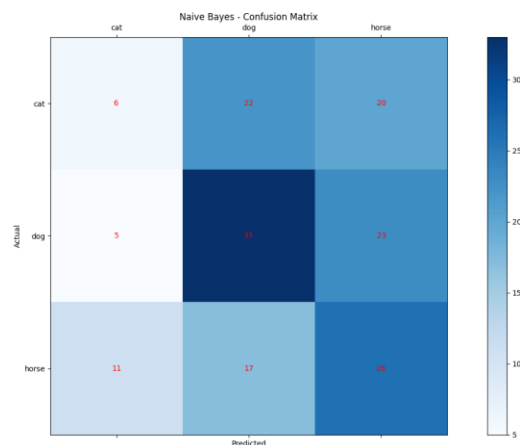


Figure 9: Confusion Matrices for Naive Bayes

The Decision Tree model performs better than Naive Bayes, with more correct predictions, especially for the dog class. However, there is still noticeable confusion between visually similar classes such as cats and dogs. While it considers all pixel values, the tree structure may not fully capture the deeper relationships within the images.

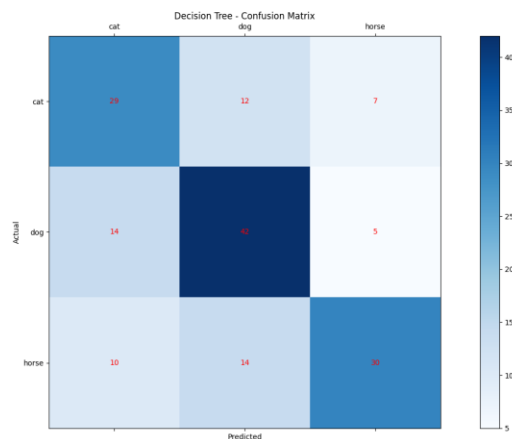


Figure 10: Confusion Matrices for Decision Tree

The MLP Classifier achieves the best results, with high accuracy across all classes and fewer misclassifications. It effectively models complex patterns in the pixel data using hidden layers and

non-linear activation functions. This allows the network to learn deeper representations and generalize better to unseen images.

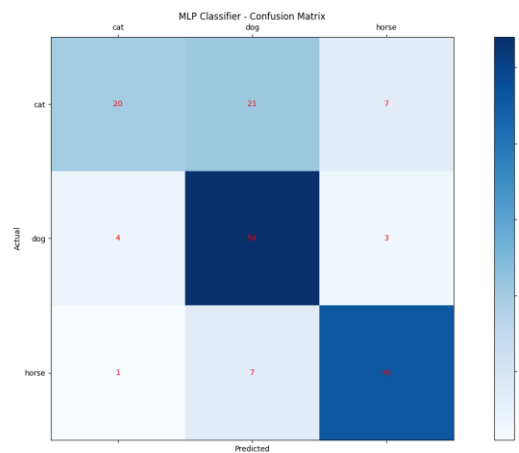


Figure 11: Confusion Matrices for MLP Classifier

We evaluated all models using the following metrics :

```
Loaded 651 images from dataset.

==== Models Metrics Summary ====
```

Model	Accuracy	Precision	Recall	F1-score
Naive Bayes	0.40	0.37	0.38	0.36
Decision Tree	0.62	0.63	0.62	0.62
MLP Classifier	0.74	0.76	0.72	0.71

Figure 12: Result

The evaluation results clearly show a performance gap between the three models. The Naive Bayes classifier achieved the lowest scores, with an accuracy of 0.40 and an F1-score of 0.36. This weak performance is due to its reliance on very simple statistical features (mean and standard deviation) and the assumption that all pixel features are independent. These assumptions do not hold true for image data, where spatial and contextual relationships between pixels are essential for correct classification.

The Decision Tree model showed a noticeable improvement, achieving an accuracy of 0.62 and an F1-score of 0.62. Unlike Naive Bayes, the Decision Tree uses all pixel intensity values as input features and builds rule-based splits to separate the classes. This approach allows it to model more complex relationships and make more informed decisions. However, it still struggles to capture deep patterns or generalize well in the presence of noise or visual similarities across classes.

The MLP Classifier outperformed both models, with the highest accuracy of 0.74 and an F1-score of 0.71. Its superior performance is due to its ability to learn non-linear and abstract representations through its multiple hidden layers and activation functions. By training on all pixel values and

applying deep learning mechanisms, MLP is capable of modeling complex structures in the image data and generalizing effectively to unseen examples. This makes it the most reliable and accurate model among the three in the context of this project.

Conclusion

This project presents a comparative study of three image classification models: Naive Bayes, Decision Tree, and MLP Classifier. The dataset consisted of 651 animal images (cats, dogs, and horses), which were preprocessed by resizing them to 64×64 pixels and flattening them into 1D vectors for model input. The models were trained and evaluated based on accuracy, precision, recall, and F1-score.

The Naive Bayes classifier, while computationally efficient and simple to implement, demonstrated the weakest performance across all evaluation metrics. Its reliance on basic statistical features such as mean and standard deviation, along with the assumption of feature independence, made it unsuitable for capturing the complex and non-linear patterns present in image data. The Decision Tree model performed better, utilizing full pixel values and providing interpretable decision paths, but it still struggled with visually similar classes. The MLP Classifier achieved the best results, thanks to its deep architecture and non-linear learning capabilities, allowing it to extract meaningful patterns and generalize well to unseen data.

In conclusion, the MLP Classifier proved to be the most effective model for this image classification task, while the Decision Tree offered a good balance between interpretability and performance. Naive Bayes served as a basic baseline.